Source of Error

- ***Round-off Error***
  - Caused by the limited number of digits that represent numbers in a computer and
  - The ways numbers are stored
  - Additions and subtractions are performed in

- ***Truncation Error***
  - Caused by approximation used in the mathematical formula of the scheme a computer

Roundoff Errors

- Roundoff errors arise because digital computers cannot represent some quantities exactly
- There are two major aspects of roundoff errors involved in numerical calculations:
  1. Digital computers have magnitude and precision limits on their ability to represent numbers.
  2. Certain numerical manipulations are highly sensitive to roundoff errors. This can result from both mathematical considerations as well as from the way in which computers perform arithmetic operations.

Computer Number Representation

- Numerical roundoff errors are directly related to the manner in which numbers are stored in a computer.
- A number system is merely a convention for representing quantities.
- We have 10 fingers and 10 toes, the number system that we are most familiar with is the decimal, or base-10.
- Numbers on the computer are represented with a binary, or base-2, system
- $(10101101)_2 = 2^7 + 2^5 + 2^3 + 2^2 + 2^0 = 128 + 32 + 8 + 4 + 1 = (173)_{10}$

Floating Point Number

- Fractional quantities are typically represented in computers using floating-point format. In this approach, which is very much like scientific notation, the number is expressed as

$$\pm s \times b^e$$

- where s = the significand (or mantissa), b = the base of the number system being used, and e = the exponent.
- For example, a value like 0.005678 could be represented in a wasteful manner as 0.005678×10^0
- However, normalization would yield 5.678×10^{-3} which eliminates the useless zeroes

Implications of Floating-Point Representation

- Let a base-10 computer with a 5-digit word size. Assume, one digit is used for the sign, two for the exponent, and two for the mantissa.
- For simplicity, assume that one of the exponent digits is used for its sign, leaving a single digit for its magnitude
- A general representation of the number following normalization would be

$$s_1 s_0 d_2 d_1 d_0 \rightarrow \qquad s_1 d_1.d_2 \times 10^{s0d0}$$

where s_0 and s_1 = the signs, d_0 = the magnitude of the exponent, and d_1 and d_2 = the magnitude of the significand digits.

Implications of Floating-Point Representation

- Largest possible value in base-10, that is, 9:
- Largest value = $\pm 9.9 \times 10^{+9}$
- Smallest value = $\pm 1.0 \times 10^{-9}$
- We could not use it to represent a quantity like Planck's constant

$$6.626 \times 10^{-34}\ J{\cdot}s$$

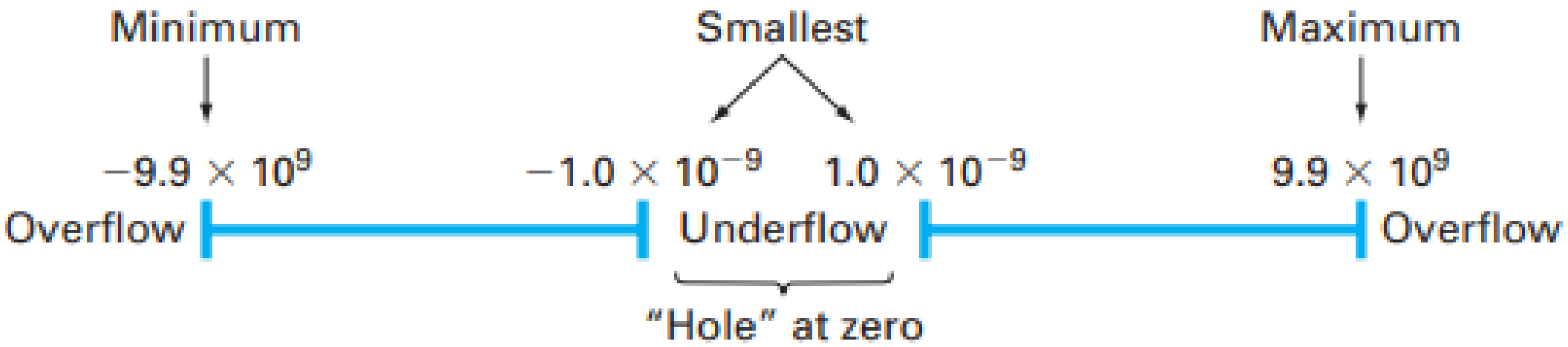
- If we increase the mantissa by one digit, the maximum/minimum value increases slightly to

$$\pm 9.99 \times 10^{\pm 9}$$

- In contrast, a one-digit increase in the exponent raises the maximum /minimum by 90 orders of magnitude to

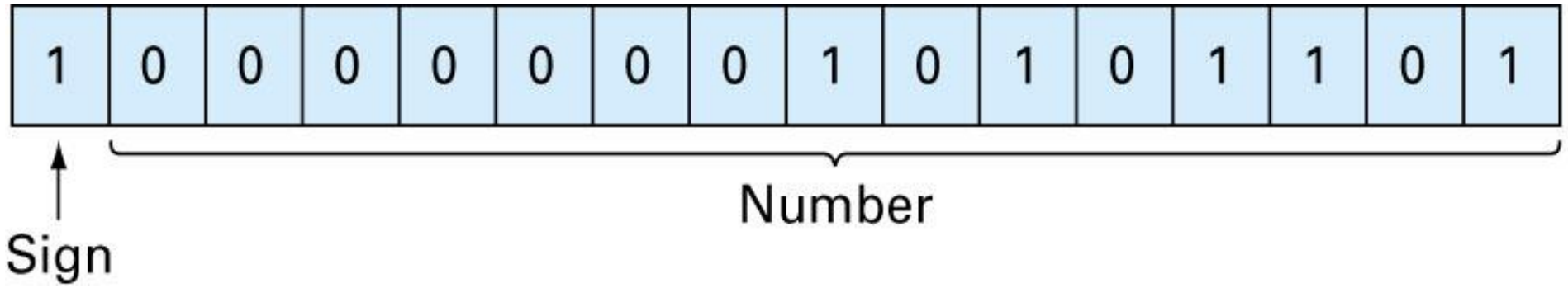
$$\pm 9.9 \times 10^{\pm 99}$$

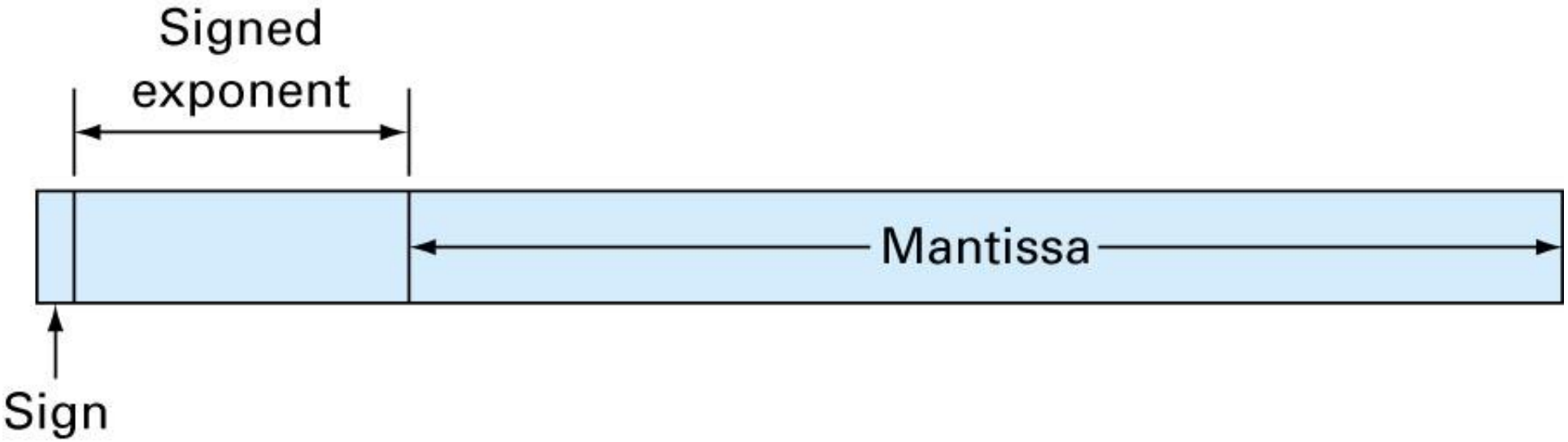
Implications of Floating-Point Representation



The number line showing the possible ranges corresponding to the hypothetical base-10 floating-point scheme described above

Computer Number Representation





IEEE-754 Floating-Point Format

	single: 8 bits double: 11 bits	single: 23 bits double: 52 bits
S	Exponent	Fraction

$$x = (-1)^s \times (1 + Fraction) \times 2^{(Exponent - Bias)}$$

- S: sign bit (0 = non-negative, 1 = negative)
- Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
  - Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
  - Significand is Fraction with the “1.(Fraction)” restored
- Exponent = Actual Exponent + Bias (or, Actual Exponent = Exponent – Bias)
- Ensures exponent is unsigned
  - Single: Bias = 127; Double: Bias = 1203

Single-Precision Range

- Exponents 00000000=0 and 11111111=255 reserved
- Smallest value
  - Exponent: 00000001=1
    Actual Exponent = 1 – 127 = –126
  - Fraction: 000…00 = significand = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Largest value
  - Exponent: 11111110=254
    Actual Exponent = 254 – 127 = +127
  - Fraction: 111…11 = significand ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Double-Precision Range

- Exponents 0000…00 = 0 and 1111…11 = 2047 reserved
- Smallest value
  - Exponent: 00000000001=1
    Actual Exponent = 1 – 1023 = –1022
  - Fraction: 000…00 = significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Largest value
  - Exponent: 11111111110=2046
    Actual Exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 = significand ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

11

Floating-Point Example

- Represent –0.625
  - $-0.625 = -0.101 = (-1)^1 \times 1.01_2 \times 2^{-1}$
  - S = 1
  - Fraction = $01000\ldots00_2$
  - Exponent = –1 + Bias
    - Single: $-1 + 127 = 126 = 01111110_2$
    - Double: $-1 + 1023 = 1022 = 01111111110_2$
- Single: 10111111001000…00
- Double: 1011111111001000…00

12

Floating Point Arithmetic in Computer

- The summation of these two numbers becomes

$$1.0_{10} + 0.00001_{10} = 1.00001_{10}$$

$$= 1.100\ 0000\ 0000\ 0000\ 0101\ 0011\ 1110\ 0010\ 1110\ 0110_2 \times 2^0$$

- Because the mantissa has 23 bits, therefore, the result of this calculation is stored in memory as,

$$0011\ 1111\ 1100\ 0000\ 0000\ 0000\ 0101\ 0011_2$$

- Thus, whenever 0.00001_{10} is added to 1.0_{10}, the result gains 0.0000000136_{10} as an error which called ***round-off error***

Floating Point Arithmetic in Computer

- When two floating-point numbers are added, the numbers are first expressed so that they have the same exponents.
- For example, if we want to add 1.557 + 0.04341, the computer would express the numbers as

  $0.1557\times10^1 + 0.004341\times10^1$.
- Then the mantissas are added to give $0.160041\times10^1=1.60041$.
- More example, suppose that we are subtracting 26.86 from 36.41. That is,

  $$0.3641\times10^2 - 0.2686\times10^2 = 0.0955\times10^2 = 9.5500$$

[Notice that the zero added to the end of the mantissa is not significant but is merely appended to fill the empty space created by the shift.]

- Even more dramatic results would be obtained when the numbers are very close as in

  $$0.7642\times10^3 - 0.7641\times10^3 = 0.0001\times10^3$$

  which would be converted to $0.1000\times10^0 = 0.1000$.
- Thus, for this case, three nonsignificant zeros are appended.
- The subtracting of two nearly equal numbers is called subtractive cancellation.

Floating Point Arithmetic in Computer

- **Large Computations:** Certain methods require extremely large numbers of arithmetic manipulations to arrive at their final results.
- A very simple case involves summing a round base-10 number that is not round in base-2. Suppose that the following function is called as:

```
int main(void){
    …;
    s_out = sum(0.0001, 10000);
    …;
}
float sum(float num, int times){
    float s = 0;
    for(int i=0; i<times; i++){
        s = s + num;
    }
    return s;
}
```

- For the case of 15 significant-digit representation

  float

  S_out = 1.000053524971008

  double

  S_out = 0.999999999999990619000

Minimizing Round-off Error

The effects of round-off error can be minimized by changing the computational algorithm although it must be devised case by case. Some useful strategies include:

- **Double precision**
- **Grouping**
- **Taylor expansions**
- **Changing definition of variables**
- **Rewriting the equation to avoid subtraction**

Minimizing Round-off Error

- **Double Precision**
- In a double precision, 8 bytes, or equivalently 64 bits, are used to store one real number. In this format 1 bit is used for sign, 11 bits are used for exponent, and 48 bits are used for mantissa.

- **Grouping**
  When the small numbers are computed, e.g. addition, subtraction, etc., grouping them helps to reduce round-off errors. For example, to add 0.00001 to unity ten thousand times one can grouped into 100 groups and each group consists of 100 small values.

Minimizing Round-off Error

- **Taylor Expansions**

as α approaches 0, accuracy of a **numerical** evaluation for

$$f(\alpha) = \frac{\sin(1+\alpha) - \sin(1)}{\alpha}$$

becomes **very poor** because round-off errors. By using Taylor expansion, we can rewrite the equation so that the accuracy for α is **improved** as

$$f(\alpha) \cong \cos(1) - 0.5\alpha^2 \sin(1)$$

Minimizing Round-off Error

- **Rewriting the equation to avoid subtraction**

  consider the equation of

$$f(x) = x\left(\sqrt{x+1} - \sqrt{x}\right)$$

  for an increasing of values *x*, the calculation of the equation above has a **loss-of-significance error**. To avoid this error one can reformulate it to get

$$f(x) = \frac{x}{\sqrt{x+1} + \sqrt{x}}$$

Truncation Error

- Numerical solutions are mostly approximations for exact solution
- Most numerical methods are based on approximating function by polynomials
- How accurately the polynomial is approximating the true function?
- Comparing the polynomial to the exact solution it becomes possible to evaluate the error, called ***truncation error***

Truncation Error

- Truncation errors are those that result from using an approximation in place of an exact mathematical procedure.
- Example 1: approximation to a derivative using a finite-difference equation:

$$\frac{dv}{dt} \approx \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$$

Truncation Error

Example 2: **The Taylor Series**

- The Taylor theorem states that any smooth function can be approximated as a polynomial.
- The Taylor series provides a means to express this idea mathematically.

$$f(x) = f(x_0) + \frac{x - x_0}{1!} f'(x_0) + \frac{(x - x_0)^2}{2!} f''(x_0) + \cdots + \frac{(x - x_0)^n}{n!} f^{(n)}(x_0) + R_n$$

Taylor Series

- Let $x=x_{i+1}$, $x_0=x_i$ and $(x_{i+1} - x_i) = h$, then the above series can be written as,

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \cdots + \frac{f^{(n)}(x_i)}{n!}h^n + R_n$$

- In general, the n_{th} order Taylor series expansion will be exact for an n_{th} order polynomial.
- In other cases, the remainder term R_n is of the order of h^{n+1}, meaning:
  - The more terms are used, the smaller the error, and
  - The smaller the spacing, the smaller the error for a given number of terms

Numerical Differentiation

- **The first order Taylor series can be used to calculate approximations to derivatives:**
  - Given: $f(x_{i+1}) = f(x_i) + f'(x_i)h + O(h^2)$
  - Then: $f'(x_i) = \dfrac{f(x_{i+1}) - f(x_i)}{h} + O(h)$
- **This is termed a "forward" difference because it utilizes data at *i* and *i*+1 to estimate the derivative.**

Numerical Differentiation

- There are also backward difference and centered difference approximations, depending on the points used:
- Forward:

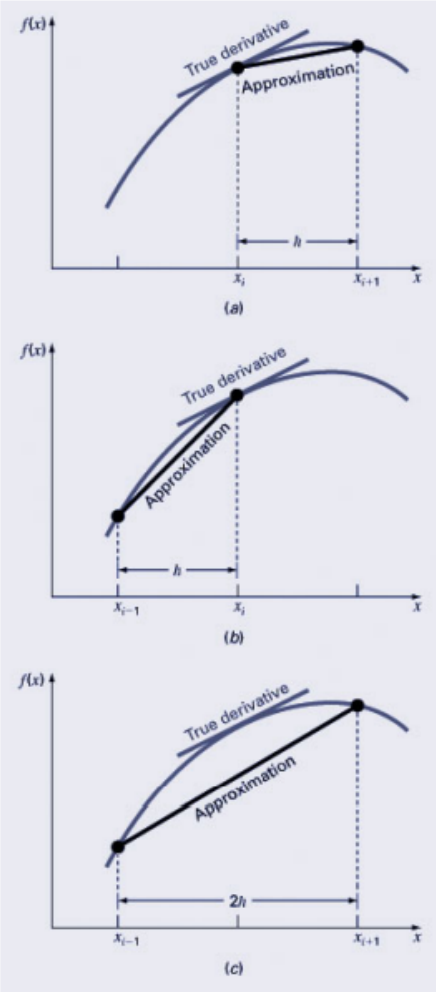
$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h} + O(h)$$

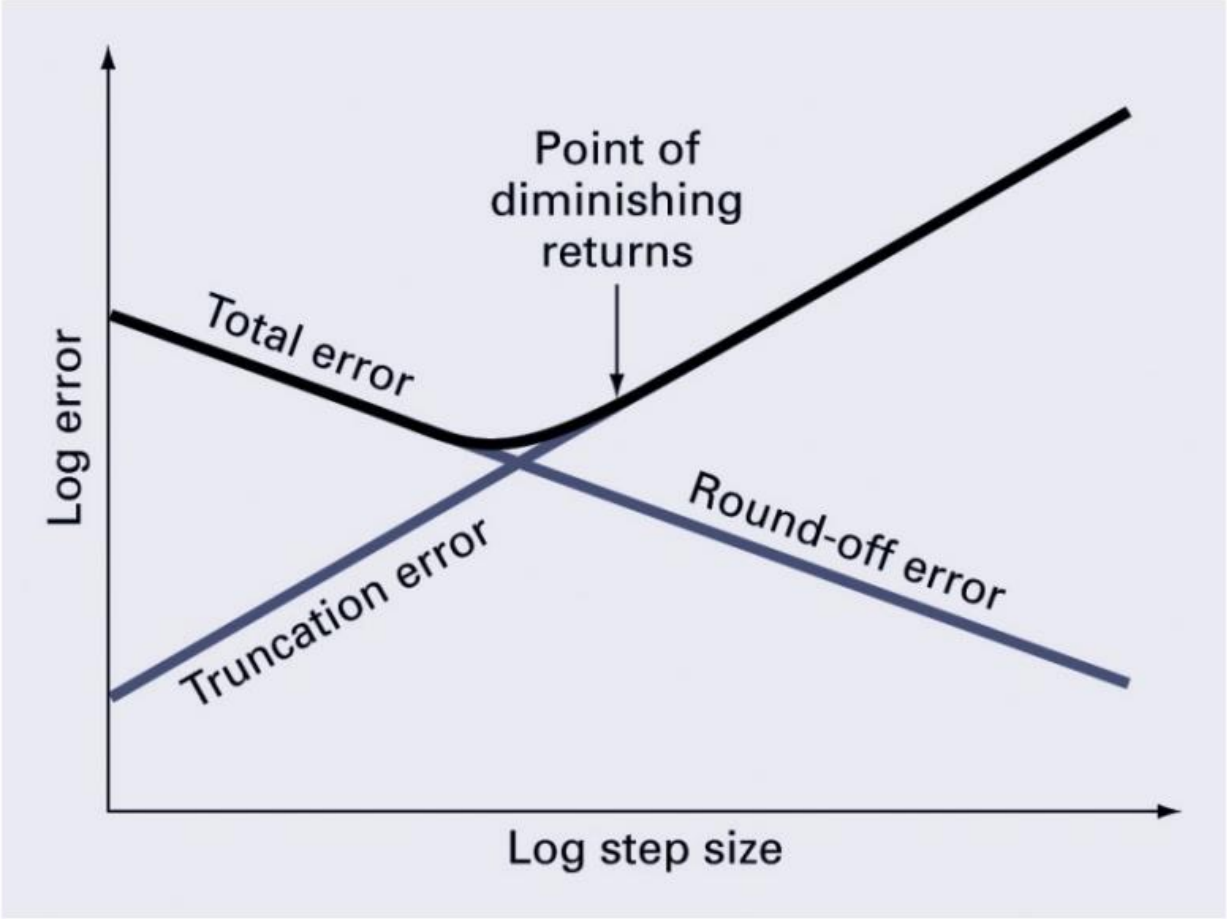
- Backward:

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{h} + O(h)$$

- Centered:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1})}{2h} + O(h^2)$$



Total Numerical Error

- The total numerical error is the summation of the truncation and roundoff errors.
- The truncation error generally increases as the step size increases, while the roundoff error decreases as the step size increases - this leads to a point of diminishing returns for step size